

Deep Leakage from Gradients

Ligeng Zhu Zhijian Liu Song Han
Massachusetts Institute of Technology
{ligeng, zhijian, songhan}@mit.edu

Abstract

Exchanging gradients is a widely used method in modern multi-node machine learning system (*e.g.*, distributed training, collaborative learning). For a long time, people believed that gradients are safe to share: *i.e.*, the training data will not be leaked by gradient exchange. However, we show that it is possible to obtain the private training data from the publicly shared gradients. We name this leakage as *Deep Leakage from Gradient* and empirically validate the effectiveness on both computer vision and natural language processing tasks. Experimental results show that our attack is much stronger than previous approaches: the recovery is *pixel-wise* accurate for images and *token-wise* matching for texts. We want to raise people’s awareness to rethink the gradient’s safety. Finally, we discuss several possible strategies to prevent such deep leakage. The most effective defense method is gradient pruning.

1 Introduction

Distributed training becomes necessary to deal with large-scale datasets. In a distributed training system, model parameters are shared across worker nodes and the updates are synchronized via passing gradients (parameter server [14, 22], all reduce [3, 29]). The distribution of computation naturally leads to the splitting of data: Each client stores a subset of the training data, and only communicates the model and the gradient. Such a framework allows to train a model using data from multiple sources without centralizing the training data [16, 17, 19, 25]. This scheme is named as collaborative learning and widely used when the data contains private information. For example, multiple hospitals can jointly train a model without sharing their patients’ medical data.

Distributed training and collaborative learning have been widely used in recent large scale machine learning. However, **does the “gradient sharing” scheme protects the training datasets of participants?** In most scenarios, people assume that gradients are safe to share and will not expose the training data. More recent studies show that gradients reveal some properties of the training data, for example, property classifier [26] (whether a data point with certain property is in the batch) and using generative adversarial networks (GAN) that can generate pictures looks similar to the training images [8, 12, 26]. Here we consider a more challenging case: can we **completely** steal the training data from gradients? Formally, given differentiable machine learning model $F()$ and its weights W , if we have the corresponding gradients ∇w w.r.t a pair of inputs and labels, can we obtain the pair of training data from gradients? Conventional wisdom would suggest that the answer is no, but we show that this is actually possible.

In this work, we demonstrate the *Deep Leakage from Gradients* (DLG): sharing the gradients will leak the private training data. We present an optimization algorithm that can obtain both the training inputs and the labels in just few iterations. To perform the attack, we first randomly generate a pair of “dummy” inputs and labels and then execute the usual forward and backward. After deriving the dummy gradients from the dummy data, instead of optimizing model weights as typical training, we optimize dummy inputs and labels to minimize the distance between dummy gradients and real

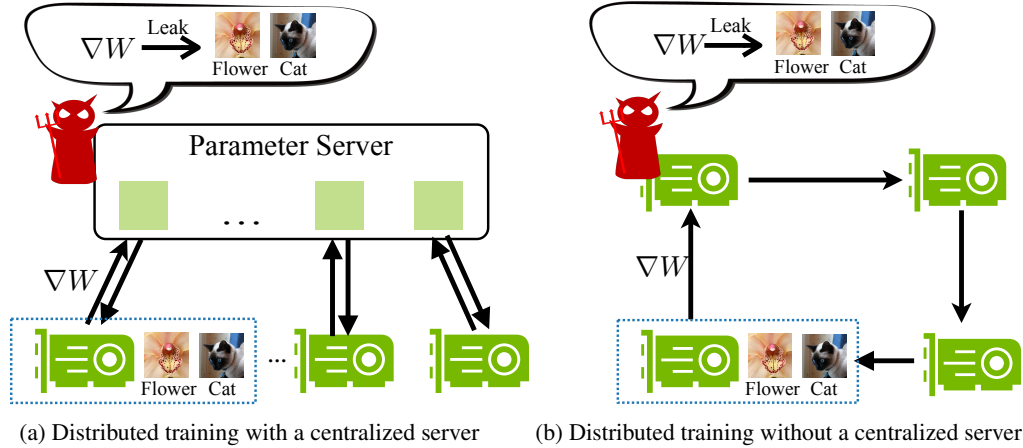


Figure 1: The deep leakage scenarios in two categories of classical multi-node training. The little red demon appears in the location where the deep leakage might happen. When performing centralized training, the parameter server is capable to steal all training data from gradients received from worker nodes. While training in a decentralized manner (e.g., ring all reduce [29]), any participant can be malicious and steal the training data from its neighbors.

gradients (illustrated in Fig. 2). When the optimization finishes, the private training data (both inputs and labels) is fully leaked. Notably, the whole process requires no extra prior about the training set.

Compared to weak attacks in previous studies (property inference [26, 32] and generative model [12] using class labels), such deep leakage from gradients is first discovered and raises people’s awareness of rethinking the safety of gradients. In centralized distributed training (Fig. 1a), parameter server, which usually does not store any training data, is able to steal local training data of all participants. For decentralized distributed training (Fig. 1b), it is even worse because any participant can steal its neighbors’ training data as long as he attempts to do that.

Compared with previous methods requires label information to generate similar images, our attack is an optimization process and does not depend on any generative models; therefore, DLG has two significant advantages. (i). DLG does not require label information, instead, can infer what the original class labels are from shared gradients. (ii). The results produced by DLG (both images and texts) are the exact original training samples instead of synthetic look-alike alternatives. We evaluate the effectiveness of our algorithm on both vision (image classification) and language tasks (masked language model). On various datasets and tasks, DLG fully recovers the training data in few gradient steps.

The deep leakage puts a challenge to the multi-node machine learning system. The fundamental gradient sharing scheme, as shown in our work, is not always reliable. To prevent the deep leakage, we demonstrate three defense strategies: gradient perturbation, half precision, and gradient compression. For gradient perturbation, we find both Gaussian and Laplacian noise with a scale higher than 10^{-2} would be a good defense. While half precision fails to protect, gradient compression successfully defends the attack with the pruned gradient is more than 20%.

Our contributions include:

- DLG can obtain the original training data from the gradients (every pixel for image and every word for texts), instead of partial properties or synthetic alternatives. To our best knowledge, DLG is the first algorithm achieving it.
- Conventional approaches usually need extra information to perform the leakage (e.g., the corresponding labels and), but DLG does not depend on a such prior and only requires the gradients.
- We analyze the attack difficulties in various settings and discuss several defense strategies against the deep leakage.

2 Related Work

2.1 Distributed Training

Training large machine learning models (e.g., deep neural networks) is computationally intensive. In order to speedup the training process, many studies worked on improving the parallelism of distributed training. There are many work improving the scalability of distributed training, both at the framework level [1, 2, 5, 28, 31] and at the algorithm level [6, 10, 15, 22, 30, 30].

In general, distributed training can be classified into two categories: with a parameter server (centralized) [14, 18, 22] and without a parameter server (decentralized) [3, 29, 31]). In both schemes, each node first performs the computation to refine its local copy of the model, and then sends gradients to other nodes. For the centralized mode, the gradients first get aggregated and then delivered back to each node. For decentralized mode, gradients are exchanged between neighboring nodes.

In many scenarios, the training data is privacy-sensitive. For example, a patient’s medical condition can not to be shared across hospitals. To avoid the sensitive information being leaked, collaborative learning has recently emerged [16, 17, 25] where two or more participants can jointly train a model while the training dataset never leave each participants’ local server. Only the gradients are shared across the network. This technique has been used to train models for medical treatments across multiple hospitals [17], analyze patient survival situations from various countries [16] and build predictive keyboards to improve typing experience [4, 19, 25].

2.2 “Shallow” Leakage from Gradients

Previous work explored that it’s possible to steal the information from gradients. For some layers, previous find the gradients already reveal certain information. For example, the embedding layer in language task only produces gradients for words occurred in training data, which reveals what words have been used in other participant’s training set [26]. But such leakage is shallow. They are not pixel-wise in vision tasks nor token-wise in language tasks. Another case is fully connected layers, where observations of gradient updates can be used to infer output feature values. However, this cannot extend to convolutional layers because the size of the features is far larger than the size of weights.

Some recent works develop learning-based methods to infer properties of the batch. They show that a binary classifier trained on gradients is able to determine whether an exact data record (membership inference [26, 32]) or a data record with certain properties (property inference [26]) is included in the other participant’s batch. Furthermore, they train GAN models [9] to synthesis images look similar to training data from the gradient [8, 12, 26], but the attack is limited and only works when all class members look alike.

3 Method

We show that it’s possible to steal an image pixel-wise and steal a sentence token-wise from the gradients. We focus on the standard synchronous distributed training: at each step t , every node i samples a minibatch $(\mathbf{x}_{t,i}, \mathbf{y}_{t,i})$ from its own dataset to compute the gradients:

$$\nabla W_{t,i} = \frac{\partial \ell(F(\mathbf{x}_{t,i}, W_t), \mathbf{y}_{t,i})}{\partial W_t} \quad (1)$$

The gradients are communicated across the network and gets averaged, which then update the weights:

$$\nabla W_t = \sum_j^C \nabla W_{t,j}; \quad W_{t+1} = W_t - \eta \nabla W_t \quad (2)$$

Given gradients $\nabla W_{t,k}$ received from other participant k , we aim to steal participant k ’s training data $(\mathbf{x}_{t,k}, \mathbf{y}_{t,k})$. $F()$ and W_t are shared by default.

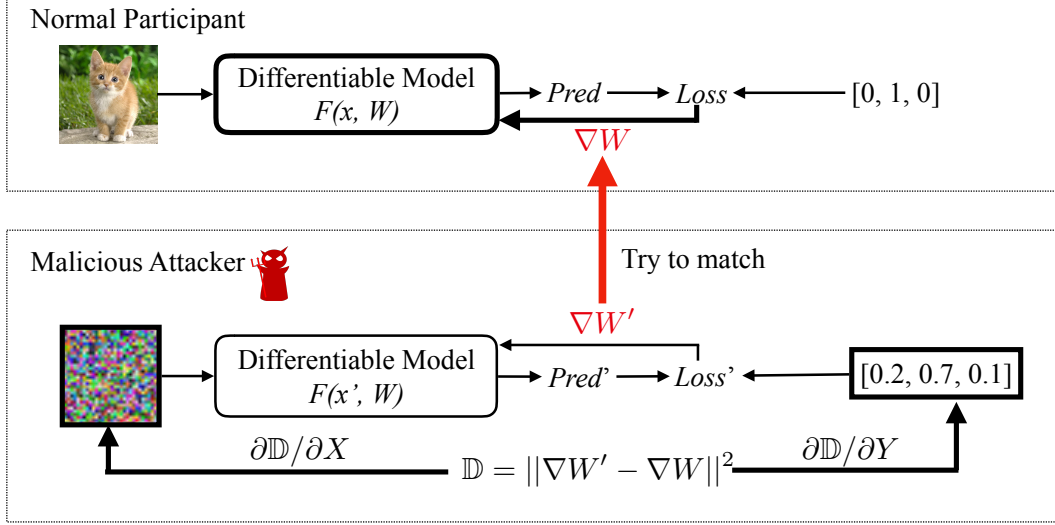


Figure 2: The overview of our DLG algorithm. Variables to be updated are marked with a bold border. While normal participants calculate ∇W to update parameter using its private training data, the malicious attacker updates its dummy inputs and labels to minimize the gradients distance. When the optimization finishes, the evil user is able to obtain the training set from honest participants.

Algorithm 1 Deep Leakage from Gradients.

Input: $F(\mathbf{x}; W)$: Differentiable model; W : parameter weights; ∇W : gradients calculated by training data

Output: private training data $\mathbf{x}, \mathbf{y}$

```

1: procedure DLG( $F, W, \nabla W$ )
2:    $\mathbf{x}'_1 \leftarrow \mathcal{N}(0, 1), \mathbf{y}'_1 \leftarrow \mathcal{N}(0, 1)$  ▷ Initialize dummy inputs and labels.
3:   for  $i \leftarrow 1$  to  $n$  do
4:      $\nabla W_t \leftarrow \partial \ell(F(\mathbf{x}'_t, W_t), \mathbf{y}'_t) / \partial W_t$  ▷ Compute dummy gradients.
5:      $\mathbb{D} \leftarrow \|\nabla W' - \nabla W\|^2$ 
6:      $\mathbf{x}'_{i+1} \leftarrow \mathbf{x}'_i - \eta \nabla_{\mathbf{x}'_i} \mathbb{D}, \mathbf{y}'_{i+1} \leftarrow \mathbf{y}'_i - \eta \nabla_{\mathbf{y}'_i} \mathbb{D}$  ▷ Update data to match gradients.
7:   end for
8:   return  $\mathbf{x}'_{n+1}, \mathbf{y}'_{n+1}$ 
9: end procedure

```

3.1 The Training Data Leakage via Gradients Matching

To recover the data from gradients, we first randomly initialize a dummy input $\mathbf{x}'$ and label input $\mathbf{y}'$. We then feed these “dummy data” into models and get “dummy gradients”.

$$\nabla W' = \frac{\partial \ell(F(\mathbf{x}', W), \mathbf{y}')}{\partial W} \quad (3)$$

Optimizing the dummy gradients close as to original also makes the dummy data close to the real training data (the trends shown in Fig. 5. Given gradients at a certain step, we obtain the training data by minimizing the following objective

$$\mathbf{x}'^*, \mathbf{y}'^* = \arg \min_{\mathbf{x}', \mathbf{y}'} \|\nabla W' - \nabla W\|^2 = \arg \min_{\mathbf{x}', \mathbf{y}'} \left\| \frac{\partial \ell(F(\mathbf{x}', W), \mathbf{y}')}{\partial W} - \nabla W \right\|^2 \quad (4)$$

The distance $\|\nabla W' - \nabla W\|^2$ is differentiable w.r.t dummy inputs $\mathbf{x}'$ and labels $\mathbf{y}'$ can thus can be optimized using standard gradient-based methods. Note that this optimization requires 2^{nd} order derivatives. We make a mild assumption that F is twice differentiable, which holds for the majority of modern machine learning models (e.g., most neural networks) and tasks.

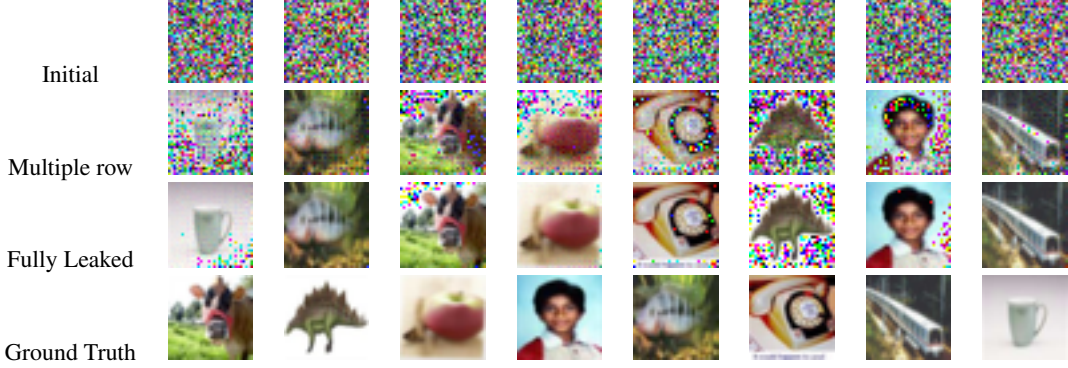


Figure 3: Results of deep leakage of batched data. Though the order may not be the same, DLG still produces images very close to the original ones.

3.2 Deep Leakage for Batched Data

The algo. 1 works well when there is only a single pair of input and label in the batch. However when we naively apply it to the case where batch size $N \geq 1$, the algorithm would be too slow to converge. We think the reason is that same batched data can have many different permutations; therefore there are $N!$ satisfactory solutions which make optimizer hard to choose gradient directions. To force the optimization closer to a solution, instead of updating the whole batch, we update a single training sample instead. We modify the *line 6* in algo. 1 to :

$$\begin{aligned} \mathbf{x}_{t+1}^{i \bmod N} &\leftarrow \mathbf{x}_t^{i \bmod N} - \nabla_{\mathbf{x}_{t+1}^{i \bmod N}} \mathbb{D} \\ \mathbf{y}_{t+1}^{i \bmod N} &\leftarrow \mathbf{y}_t^{i \bmod N} - \nabla_{\mathbf{y}_{t+1}^{i \bmod N}} \mathbb{D} \end{aligned} \quad (5)$$

Then we can observe fast and stable convergence. We list the iterations required for convergence for different batch sizes in Tab. 1 and provide visualized results in Fig. 3.

	BS=1	BS=2	BS=4	BS=8
ResNet-20	270	602	1173	2711

Table 1: The iterations required for restore batched data on CIFAR [20] dataset. The deep leakage becomes harder when batch size increases.

4 Experiments

Setup. Implementing algorithm. 1 requires to calculate the high order gradients and we choose PyTorch [28] as our experiment platform. We use L-BFGS [24] with learning rate 1, history size 100 and max iterations 20 and optimize for 1200 iterations and 100 iterations for image and text task respectively. More task-specific details can be found in following sub-sections.

4.1 Deep Leakage on Image Classification

Given an image containing objects, images classification aims to determine the class of the item. We experiment our algorithm on modern CNN architectures ResNet-56 [11] and pictures from MNIST [21], CIFAR [20], SVHN [27] and LFW [13]. One change we have made to the models is replacing activation ReLU to Sigmoid, as our algorithm requires the model to be twice-differentiable. For image labels, instead of directly optimizing the discrete categorical values, we random initialize a vector with shape $N \times C$ where N is the batch size and C is the number of classes, and then take its softmax output as the one-hot label for optimization.

The leaking process is visualized in Fig. 4. We start with random Gaussian noise (first column) and try to match the gradients produced by the dummy data and real ones. As shown in Fig 5, minimizing

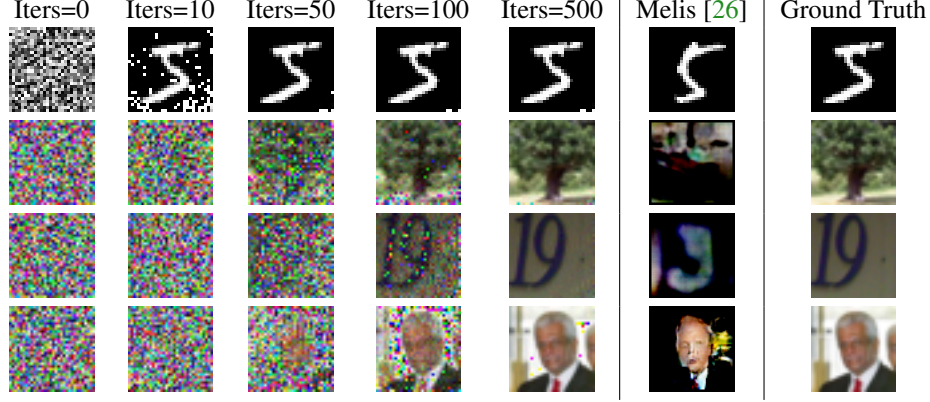


Figure 4: The visualization showing the deep leakage on images from MNIST [21], CIFAR [20], SVHN [27] and LFW [13] respectively. Our algorithm fully recovers the four images while previous work only succeed on images with clean background.

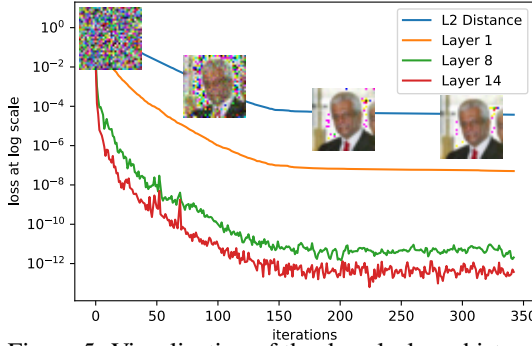


Figure 5: Visualization of the deep leakage history. During the optimization, the gradients’ distance gets smaller. The MSE between leaked image and the original image also gets smaller.

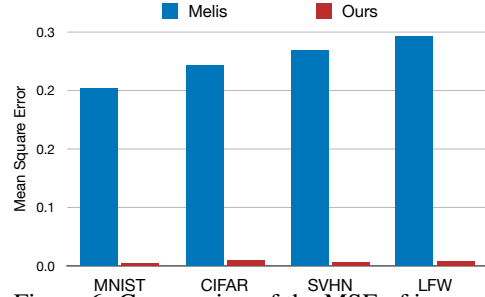


Figure 6: Comparison of the MSE of images leaked by different algorithms and the ground truth. Our method consistently outperforms previous approach by a large margin.

the distance between gradients also reduces the gap between data. We observe that monochrome images with a clean background (MNIST) are easiest to recover, while complex images like face take more iterations to recover (Fig. 4). When the optimization finishes, the recover results are almost identical to ground truth images, despite few negligible artifact pixels.

We visually compare the results from other method [26] and ours in Fig. 4. The previous method uses GAN models when the class label is given and only works well on MNIST. The result on SVHN, though is still visually recognizable as text “9”, this is no longer the original training image. The cases are even worse on LFW and collapse on CIFAR. We also make a numerical comparison by performing leaking and measuring the MSE on all dataset images in Fig. 6. Images are normalized to the range $[0, 1]$ and our algorithm appears much better results (ours < 0.03 v.s. previous > 0.2) on all four datasets.

4.2 Deep Leakage on Masked Language Model

For language task, we verify our algorithm on Masked Language Model (MLM) task. In each sequence, 15% of the words are replaced with a [MASK] token and MLM model attempts to predict the original value of the masked words from a given context. We choose BERT [7] as our backbone and adapt hyperparameters from the official implementation^{*}.

Different from vision tasks where inputs are continuous RGB values, language models preprocess discrete words into embeddings. We apply our algorithm on embeddings and minimize the gradients

^{*}<https://github.com/google-research/bert>

	Example 1	Example 2	Example 3
Initial Sentence	tilting fill given **less word **itude fine **nton over- heard living vegas **vac **vation *f forte **dis ce- rambycidae ellison **don yards marne **kali	toni **enting asbestos cut- ler km nail **oof **dation **ori righteous **xie lucan **hot **ery at **tle ordered pa **eit smashing proto	[MASK] **ry toppled **wled major relief dive displaced **lice [CLS] us apps _ **face **bet
Iters = 10	tilting fill given **less full solicitor other ligue shrill living vegas rider treatment carry played sculptures life- long ellison net yards marne **kali	toni **enting asbestos cutter km nail undefeated **dation hole righteous **xie lucan **hot **ery at **tle ordered pa **eit smashing proto	[MASK] **ry toppled iden- tified major relief gin dive displaced **lice doll us apps _ **face space
Iters = 20	registration , volunteer ap- plications , at student travel application open the ; week of played ; child care will be glare .	we welcome proposals for tutor **ials on either core machine denver softly or topics of emerging impor- tance for machine learning .	one **ry toppled hold major ritual ' dive annual confer- ence days 1924 apps novel- ist dude space
Iters = 30	registration , volunteer ap- plications , and student travel application open the first week of september . child care will be available .	we welcome proposals for tutor **ials on either core machine learning topics or topics of emerging impor- tance for machine learning .	we invite submissions for the thirty - third annual con- ference on neural informa- tion processing systems .
Original Text	Registration, volunteer applications, and student travel application open the first week of September. Child care will be available.	We welcome proposals for tutorials on either core machine learning topics or topics of emerging importance for machine learning.	We invite submissions for the Thirty-Third Annual Conference on Neural Information Processing Systems.

Table 2: The progress of deep leakage on language tasks.

distance between the one produced dummy embeddings and real ones. After optimization finishes, we derive original text by finding the closest entry the embedding matrix.

In Tab. 2, we exhibit the leaking history on three sentences selected from NeurIPS conference page. Similar to the vision task, we start with randomly initialized embedding: the reverse query results at iteration 0 is meaningless. During the optimization, the gradients produced by dummy embedding gradually match the original ones and so the embeddings. In later iterations, part of sequence gradually appears. In example 3, at iteration 20, ‘annual conference’ appeared and at iteration 30, we have a sentence very similar original text. Though few mismatches caused by the ambiguity in tokenizing, the main content of original training data is already leaked.

5 Defense Strategies

5.1 Noisy Gradients

One straightforward attempt to defense DLG is to add noise on gradients before sharing. To evaluate, we experiment Gaussian and Laplacian noise (widely used in differential privacy studies) distributions with variance range from 10^{-1} to 10^{-4} and central 0. From Fig. 7a and 7b, we observe that the defense effect mainly depends on the magnitude of distribution variance and less related to the noise types. When variance is at the scale of 10^{-4} , the noisy gradients do not prevent the leak. For noise with variance 10^{-3} , though with artifacts, the leakage can still be performed. Only when the variance is larger than 10^{-2} and the noise is starting affect the accuracy, DLG will fail to execute. We also notice that Laplacian tends to slightly a better defense when both at scale 10^{-3} .

Another common perturbation on gradients is half precision, which was initially designed to save memory footprints and widely used to reduce communication bandwidth. We test two popular half precision implementations IEEE float16 (*Single-precision floating-point format*) and bfloat16 (*Brain Floating Point* [33], a truncated version of 32 bit float). Shown in Fig. 7c, unfortunately, both half precision failed to protect the training data.

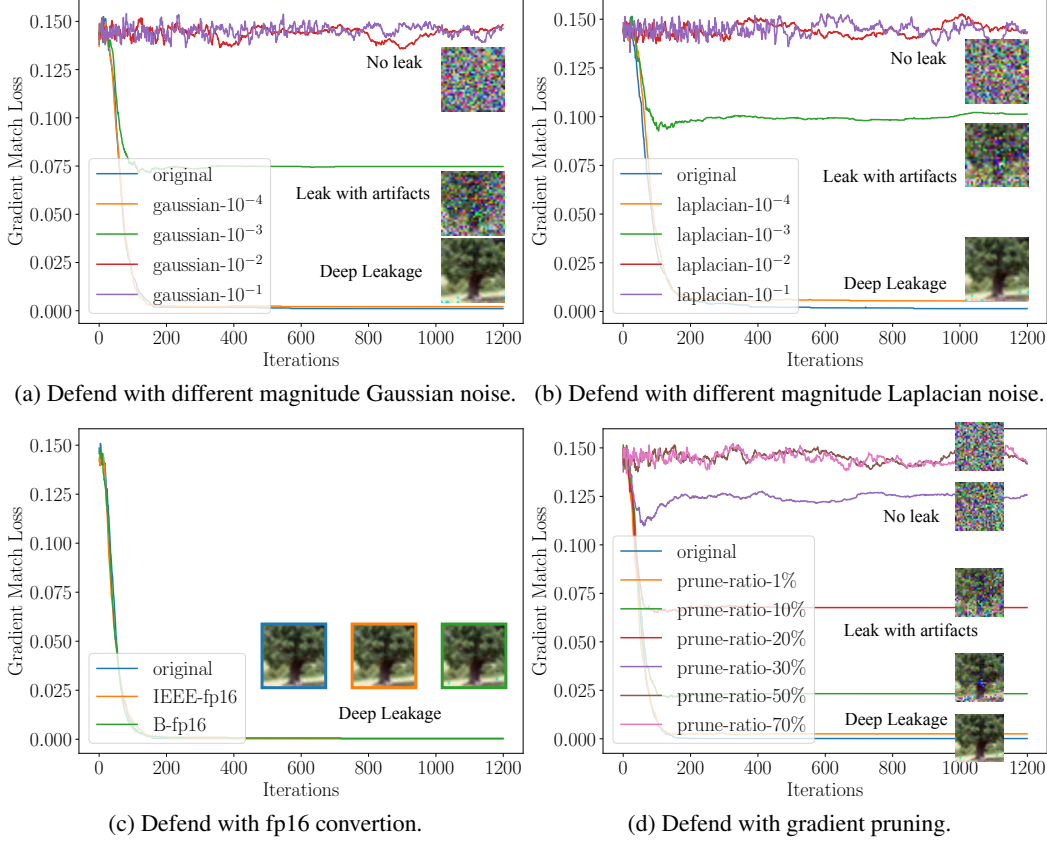


Figure 7: The effectiveness of various defense strategies.

5.2 Gradient Compression and Sparsification

We also experimented to defend by gradient compression [23, 34]. Gradient compression prunes small gradients to zero, therefore it’s more difficult for DLG to match the gradients since the optimization target also gets pruned. We evaluate how different level of sparsities (range from 1% to 70%) defense the leakage. When sparsity is 1% to 10%, it has almost no effects against DLG. When prune ratio increases to 20%, as shown in Fig. 7d, there are obvious artifact pixels on the recover images. We notice that maximum tolerance of sparsity if around 20%. When pruning ratio is larger than 20%, the recovered images are no longer visually recognizable and thus gradient compression successfully prevents the leakage.

Previous work [23, 34] show that gradients can be compressed by more than $300\times$ without losing accuracy. In which case, the sparsity is above 99% and already exceeds the maximum tolerance of DLG (which is around 20%). It suggests that compressing the gradients can be a good practical approach to avoid the leakage.

6 Conclusions

In this paper, we introduce the *Deep Leakage from Gradients*: an algorithm that can obtain the local training data from public shared gradients. Our method does not rely on any generative model or extra prior about the data and can recover the exact training pair in few iteration steps. This sets a challenge to distributed training and brings us to rethink about the safety of exiting gradients sharing scheme. Our experiments on vision and language tasks both demonstrate the critical risks of this deep leakage. To prevent this, we propose gradient perturbation, half precision and gradient compression and compare their defense effectiveness.

Acknowledgments

We sincerely thank MIT Quest for Intelligence, MIT-IBM Watson AI lab, MIT-Sensetime Alliance and Facebook for supporting this work. We thank Google for providing the computation resource.

References

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org. 3
- [2] Takuya Akiba, Keisuke Fukuda, and Shuji Suzuki. ChainerMN: Scalable Distributed Deep Learning Framework. In *Proceedings of Workshop on ML Systems in The Thirty-first Annual Conference on Neural Information Processing Systems (NIPS)*, 2017. 3
- [3] Blaise Barney et al. Introduction to parallel computing. 1, 3
- [4] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloe Kiddon, Jakub Konecny, Stefano Mazzocchi, H Brendan McMahan, et al. Towards federated learning at scale: System design. *arXiv preprint arXiv:1902.01046*, 2019. 3
- [5] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems. *arXiv preprint arXiv:1512.01274*, 2015. 3
- [6] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Andrew Senior, Paul Tucker, Ke Yang, Quoc V Le, et al. Large scale distributed deep networks. In *Advances in neural information processing systems*, pages 1223–1231, 2012. 3
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018. 6
- [8] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. Model inversion attacks that exploit confidence information and basic countermeasures. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pages 1322–1333. ACM, 2015. 1, 3
- [9] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in neural information processing systems*, pages 2672–2680, 2014. 3
- [10] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch sgd: Training imagenet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017. 3
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 5
- [12] Briland Hitaj, Giuseppe Ateniese, and Fernando Pérez-Cruz. Deep models under the GAN: information leakage from collaborative deep learning. *CoRR*, abs/1702.07464, 2017. 1, 2, 3
- [13] Gary B. Huang, Manu Ramesh, Tamara Berg, and Erik Learned-Miller. Labeled faces in the wild: A database for studying face recognition in unconstrained environments. Technical Report 07-49, University of Massachusetts, Amherst, October 2007. 5, 6
- [14] Forrest N Iandola, Matthew W Moskewicz, Khalid Ashraf, and Kurt Keutzer. Firecaffe: near-linear acceleration of deep neural network training on compute clusters. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2592–2600, 2016. 1, 3
- [15] Xianyan Jia, Shutao Song, Wei He, Yangzihao Wang, Haidong Rong, Feihu Zhou, Liqiang Xie, Zhenyu Guo, Yuanzhou Yang, Liwei Yu, et al. Highly scalable deep learning training system with mixed-precision: Training imagenet in four minutes. *arXiv preprint arXiv:1807.11205*, 2018. 3
- [16] Arthur Jochems, Timo M Deist, Issam El Naqa, Marc Kessler, Chuck Mayo, Jackson Reeves, Shruti Jolly, Martha Matuszak, Randall Ten Haken, Johan van Soest, et al. Developing and validating a survival prediction model for nscl patients through distributed learning across 3 countries. *International Journal of Radiation Oncology* Biology* Physics*, 99(2):344–352, 2017. 1, 3
- [17] Arthur Jochems, Timo M Deist, Johan Van Soest, Michael Eble, Paul Bulens, Philippe Coucke, Wim Dries, Philippe Lambin, and Andre Dekker. Distributed learning: developing a predictive model based on data from multiple hospitals without data leaving the hospital—a real life proof of concept. *Radiotherapy and Oncology*, 121(3):459–467, 2016. 1, 3

- [18] Hanjoo Kim, Jaehong Park, Jaehee Jang, and Sungroh Yoon. Deepspark: Spark-based deep learning supporting asynchronous updates and caffe compatibility. *arXiv preprint arXiv:1602.08191*, 2016. 3
- [19] Jakub Konečný, H. Brendan McMahan, Felix X. Yu, Peter Richtarik, Ananda Theertha Suresh, and Dave Bacon. Federated learning: Strategies for improving communication efficiency. In *NIPS Workshop on Private Multi-Party Machine Learning*, 2016. 1, 3
- [20] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, Citeseer, 2009. 5, 6
- [21] Yann LeCun. The mnist database of handwritten digits. <http://yann.lecun.com/exdb/mnist/>. 5, 6
- [22] Mu Li, David G Andersen, Jun Woo Park, Alexander J Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J Shekita, and Bor-Yiing Su. Scaling distributed machine learning with the parameter server. In *11th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 14)*, pages 583–598, 2014. 1, 3
- [23] Yujun Lin, Song Han, Huizi Mao, Yu Wang, and William J Dally. Deep gradient compression: Reducing the communication bandwidth for distributed training. *arXiv preprint arXiv:1712.01887*, 2017. 8
- [24] Dong C Liu and Jorge Nocedal. On the limited memory bfgs method for large scale optimization. *Mathematical programming*, 45(1-3):503–528, 1989. 5
- [25] H Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, et al. Communication-efficient learning of deep networks from decentralized data. *arXiv preprint arXiv:1602.05629*, 2016. 1, 3
- [26] Luca Melis, Congzheng Song, Emiliano De Cristofaro, and Vitaly Shmatikov. Exploiting unintended feature leakage in collaborative learning. *CoRR*, abs/1805.04049, 2018. 1, 2, 3, 6
- [27] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y Ng. Reading digits in natural images with unsupervised feature learning. 2011. 5, 6
- [28] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. In *NIPS-W*, 2017. 3, 5
- [29] Pitch Patarasuk and Xin Yuan. Bandwidth optimal all-reduce algorithms for clusters of workstations. *Journal of Parallel and Distributed Computing*, 69(2):117–124, 2009. 1, 2, 3
- [30] Benjamin Recht, Christopher Re, Stephen Wright, and Feng Niu. Hogwild: A lock-free approach to parallelizing stochastic gradient descent. In *Advances in neural information processing systems*, pages 693–701, 2011. 3
- [31] Alexander Sergeev and Mike Del Balso. Horovod: fast and easy distributed deep learning in tensorflow. *arXiv preprint arXiv:1802.05799*, 2018. 3
- [32] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 3–18. IEEE, 2017. 2, 3
- [33] Giuseppe Tagliavini, Stefan Mach, Davide Rossi, Andrea Marongiu, and Luca Benini. A transprecision floating-point platform for ultra-low power computing. *CoRR*, abs/1711.10374, 2017. 7
- [34] Yusuke Tsuzuku, Hiroto Imachi, and Takuya Akiba. Variance-based gradient compression for efficient distributed deep learning. *arXiv preprint arXiv:1802.06058*, 2018. 8